Vectors Part 1

Instructor: Dr. Khaled Mohamad

In linear algebra, a vector is an ordered list of numbers.

In Python, vectors can be represented using various data types. While lists might appear to be the simplest option for representing a vector, they are not suitable for many linear algebra operations. Consequently, it is generally more effective to use NumPy arrays for creating vectors. Below are four methods to create a vector using NumPy

In [1]:
```
#Import numpy and Matplotlib modules
import numpy as np
import matplotlib.pyplot as plt

#ignore warning message
import warnings
warnings.filterwarnings('ignore')
```

In [7]:
```
List = [1,2,3]
List
```

Out[7]:
```
[1, 2, 3]
```

In [8]:
```
#1D Array
Ar = np.array([1,2,3])
Ar
```

Out[8]:
```
array([1, 2, 3])
```

In [9]:
```
# Row
row = np.array([ [1,2,3] ])
row
```

Out[9]:
```
array([[1, 2, 3]])
```

In [10]:
```
#Column
col = np.array([ [1],[2],[3] ])
col
```

Out[10]:
```
array([[1],
       [2],
       [3]])
```

Focus on how arrays, rows, and columns are represented in NumPy functions. It's crucial to differentiate between them.

Orientation and shape

The variable `Ar = np.array([1,2,3])` is a one-dimensional array in NumPy, meaning it does not have a specific orientation as a row or column vector. **In NumPy**, orientation is determined by the use of brackets: the outermost brackets enclose all the numbers into a single entity. Each additional set of brackets signifies a row. For instance, a row vector (row) contains all numbers in a single row, whereas a column vector (col) consists of multiple rows, each containing one number.

To understand these orientations better, we can examine the shapes of these variables, which is often a helpful practice during coding:

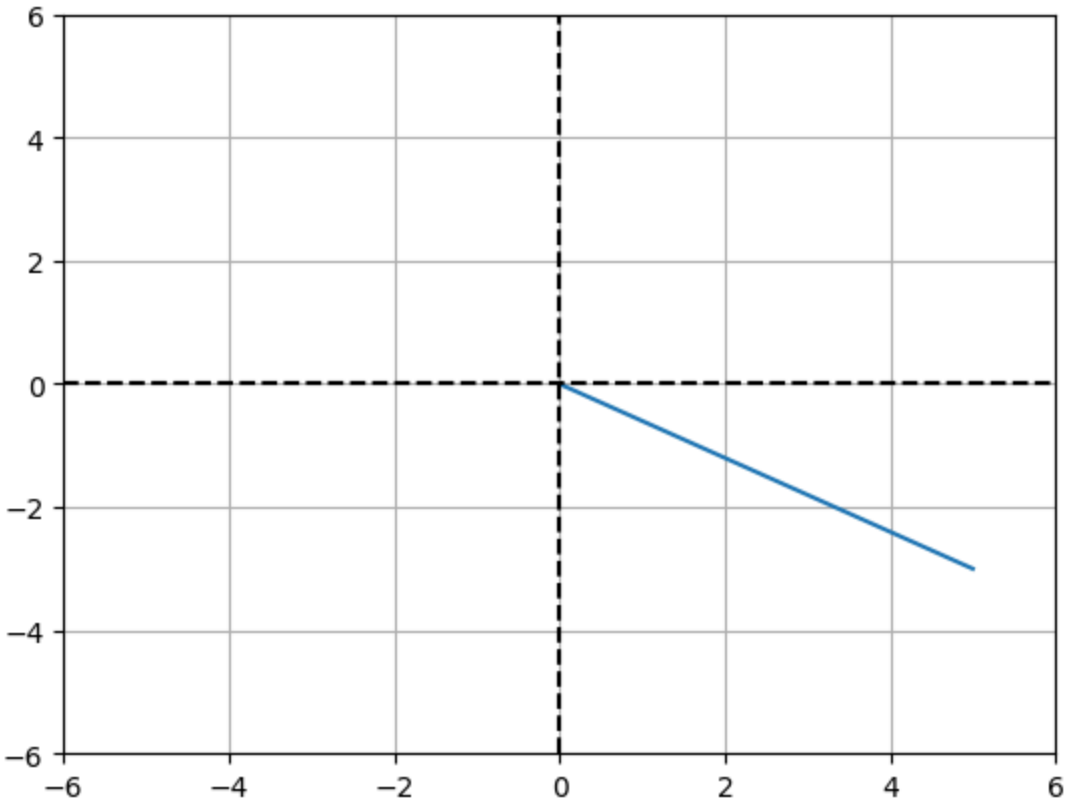
In [14]:
```
print(f'List: {np.shape(List)}')
print(f'Array: {Ar.shape}')
print(f'Row: {row.shape}')
print(f'Column: {col.shape}')
```

```
List: (3,)
Array: (3,)
Row: (1, 3)
Column: (3, 1)
```

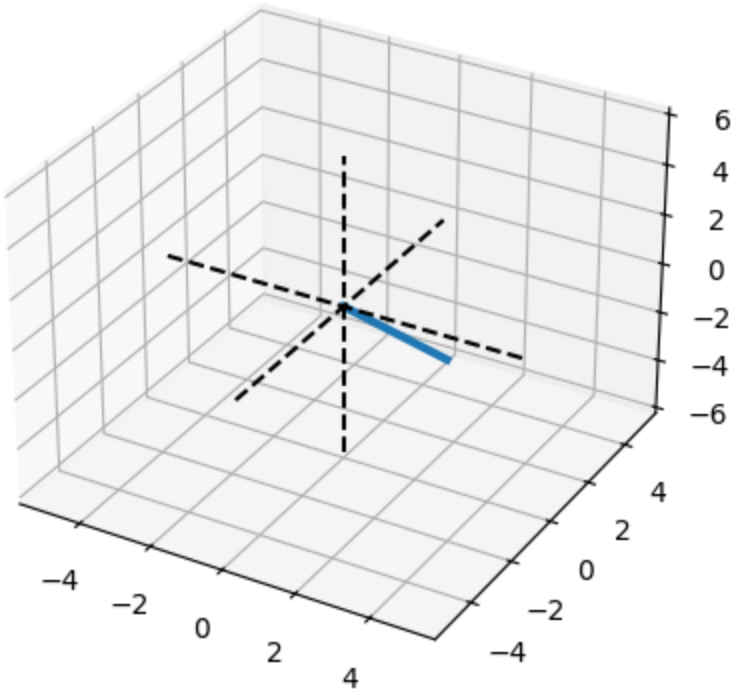
Dimensions are always listed as (rows,columns). The output of size (3) is number or a list of number, which is 1D array

Geometric of vectors

In [17]:
```
# A 2-dimensional vector
v2 = [ 5, -3 ]

# plot v2 in 2D
plt.plot([0,v2[0]],[0,v2[1]])

# The following could be confusing. Remember that, here, you putting dashed axise (
plt.plot([-6, 6],[0, 0],'k--') #dashed line in x direction
plt.plot([0, 0],[-6, 6],'k--') #dashed line in y direction
plt.grid()
plt.axis((-6, 6, -6, 6))  #maximum and minum numbers in x and y axis. First two ia
plt.show()
```



In [39]:
```
# A 3-dimensional vector
v3 = [ 5, -4, 3 ]

# plot the 3D vector
fig = plt.figure(figsize=plt.figaspect(1)) #aspect is zooming
ax = plt.axes(projection = '3d')
ax.plot([0, v3[0]],[0, v3[1]],[0, v3[2]],linewidth=3)

# 3D dashed axes inside the 3D cont. to be easy for visualization
ax.plot([0, 0],[-5, 5],[0, 0],'k--') #x
ax.plot([0, 0],[0, 0],[-6, 6],'k--')#y
ax.plot([-5, 5],[0, 0],[0, 0],'k--') #z
plt.show()
```



Vector Addition and Subtraction

In [41]:
```
a = np.array([4,5,6])
b = np.array([10,20,30])
c = np.array([0,3,6,9])
```

In [43]:
```
a+b
```

Out[43]:
```
array([14, 25, 36])
```

In [45]:
```
a+c
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[45], line 1
----> 1 a+c

ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

You got error because of dimension inequality

What about the orientation (row and column)

In linear algebra, adding vector with different orientation is not defined operation but in Python (Numpy) is possible because it is implemented an operation called broadcasting. We will learn more about broadcasting later.

In [45]:
```
#recall
print(f' a: {a}')
```

```
a: [4 5 6]
```

In [32]:
```
#Applying transpose function. It convert row into column and vice versa. Remmbmber,
b_col = np.array([[12, 13, 14]]).T
b_col
```

Out[32]:
```
array([[12],
       [13],
       [14]])
```

Another way to change row to column (or vice-versa) is using b_col = np.transpose(v)

In [47]:
```
a+b_col
```

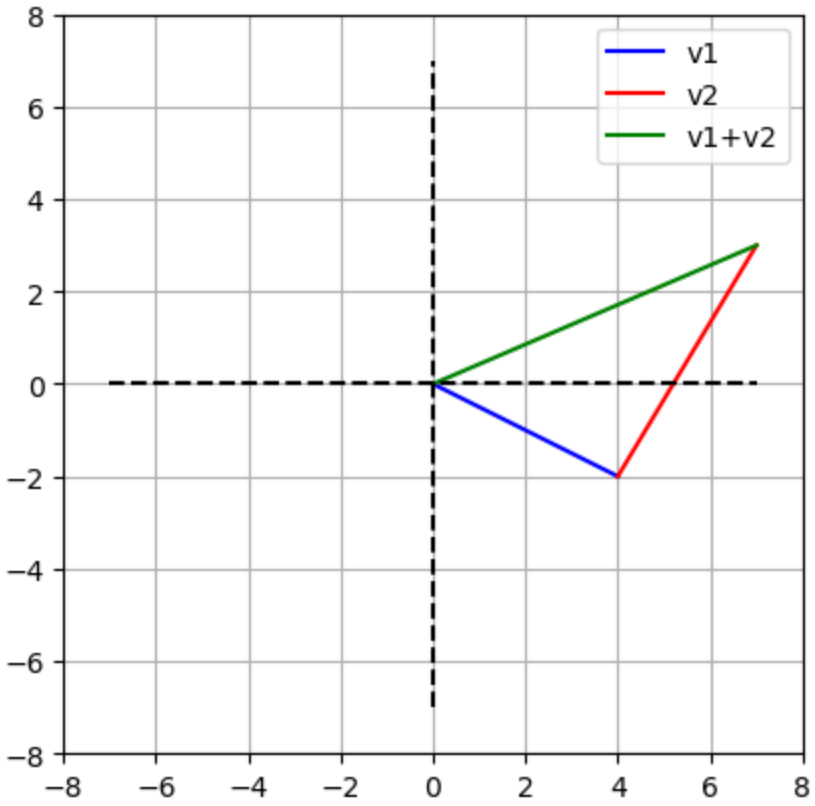
Out[47]:
```
array([[16, 17, 18],
       [17, 18, 19],
       [18, 19, 20]])
```

Important Note: two vectors can be added together only if they have the same dimensionality and the same orientation.

Geometric of vectors addition

In [148…
```
# Identifying two vectors in 2D
v1 = np.array([ 4, -2 ])
v2 = np.array([ 3,  5 ])

v3 = v1 + v2

print(f'The resultant vector of v1 + v2 is v3 = {v3}')


# plot them (2D)
plt.plot([0, v1[0]],[0, v1[1]],'b',label='v1')
plt.plot([0, v2[0]]+v1[0],[0, v2[1]]+v1[1],'r',label='v2')
plt.plot([0, v3[0]],[0, v3[1]],'g',label='v1+v2')


plt.plot([-7, 7],[0, 0],'k--') #dashed line in x direction
plt.plot([0, 0],[-7, 7],'k--') #dashed line in x direction


plt.legend()
plt.axis('square')
plt.axis((-8, 8, -8, 8 ))
plt.grid()
plt.show()
```

```
The resultant vector of v1 + v2 is v3 = [7 3]
```


v1
v2
v1+v2
8
6
4
2
0
−2
−4
−6
−8
−8
−6
−4
−2
0
2
4
6
8